PostgreSQL Major and Minor Version Upgrade: Strategy, Safety, and Execution — PostgreSQL Foundations Series

Wayne C.
Published Oct 27, 2025

+ Follow

Upgrading PostgreSQL is not just a version bump — it's a critical operation that affects data integrity, application compatibility, security posture, and performance. Understanding the difference between **major** and **minor** upgrades and how to approach them safely is

This guide walks you through:

- 🔹 The distinction between major and minor upgrades
- 🔹 Recommended methods for both
- 🔹 Pre-upgrade checks
- 🔹 Tools and procedures
- 🔹 Real-world strategies for zero-downtime migrations

🧭 **Understanding PostgreSQL Versioning**

PostgreSQL uses **semantic versioning** in the form: X.Y

- X = Major version (e.g., 15, 16)
- Y = Minor version (e.g., 16.2, 16.3)

🔹 **Minor Version Upgrades:**

- Patch-level updates (security, bug fixes)
- No catalog changes
- **In-place, backward compatible**
- Safe and quick to apply

🔸 **Major Version Upgrades:**

- Structural/internal changes
- Catalog version changes
- **Requires full migration**
- May remove deprecated features

🔹 **Minor Version Upgrade**

Minor upgrades can be applied with package managers (apt, dnf, yum, pacman).

🔹 **On Ubuntu/Debian:**

```
sudo apt update
sudo apt install postgresql-16
```

🔹 **On RHEL/CentOS:**

```
sudo dnf upgrade postgresql16*
```

🧪 **Steps:**

1. Take a **base backup** or snapshot.
2. Review release notes for any config deprecations.
3. Apply the update.
4. Restart PostgreSQL:

💡 **Pro Tip:** Always upgrade to the latest minor version within your major series to patch security vulnerabilities and bugs.

🔸 **Major Version Upgrade**

Major upgrades are more involved and require:

- Data directory migration
- Testing application compatibility
- Extension and plugin validation

Methods:

1. **pg_upgrade** (recommended)
2. pg_dump / pg_restore
3. Logical replication (zero-downtime option)

🧰 **Using pg_upgrade for Major Upgrade**

pg_upgrade enables fast upgrades by reusing the data files and only migrating the metadata/catalog.

✅ **Step-by-step:**
1. Install the new PostgreSQL version (side-by-side):

```
sudo apt install postgresql-17
```

2. Stop both old and new PostgreSQL services.
3. Run pg_upgrade:

```
sudo -u postgres /usr/lib/postgresql/17/bin/pg_upgrade \
  --old-datadir=/var/lib/postgresql/16/main \
  --new-datadir=/var/lib/postgresql/17/main \
  --old-bindir=/usr/lib/postgresql/16/bin \
  --new-bindir=/usr/lib/postgresql/17/bin \
  --check
```

If check passes, perform the actual upgrade:

```
sudo -u postgres /usr/lib/postgresql/17/bin/pg_upgrade \
  --jobs 4 --link
```

4. Run post-upgrade cleanup:

```
./analyze_new_cluster.sh
./delete_old_cluster.sh
```

🔄 **Alternative: pg_dump / pg_restore**

Use when:

- You're skipping multiple versions
- You want to change storage layout
- You're migrating between OS or architectures

```
pg_dumpall -U postgres > all.sql
```

Then install new version and restore:

```
psql -U postgres -f all.sql
```

📉 Downsides:

- Longer downtime
- More manual validation
- May need to rebuild extensions or indexes

🧊 **Zero-Downtime Option: Logical Replication**

1. Setup new PostgreSQL version
2. Create publication on old:

```
CREATE PUBLICATION my_pub FOR ALL TABLES;
```

1. On new version:

```
CREATE SUBSCRIPTION my_sub
  CONNECTION 'host=localhost dbname=mydb user=replicator password=pass'
  PUBLICATION my_pub;
```

Once caught up, switch traffic.

✅ **Before You Upgrade: Preflight Checklist**

🧪 **Post-upgrade Tasks**

- Rebuild statistics:
- Recompile extensions (CREATE EXTENSION)
- Validate application behavior
- Rotate logs and remove old data
- Benchmark performance (some settings may need tuning)

🔀 **Common Issues & Fixes**

🚀 **Tips for Success**

- Never skip testing.
- Automate backup before upgrades.
- Use pg_upgrade where possible — it's fast and proven.
- Always read the official notes: www.postgresql.org/docs/release/
- For large-scale systems, consider blue-green deployments with logical replication.

🔒 **Security Benefits of Staying Updated**

Staying current isn't just about features — it's about:

- Fixing known vulnerabilities (e.g., privilege escalations)
- Patching crash bugs
- Improving query planner and memory management

PostgreSQL is secure, but only if you keep it fresh.

🗺️ **Vision Strategy: Plan for the Future**

🎯 Stick to **latest 2 versions** for best support and features.

🧭 **Final Summary**

🧠 **Conclusion**

Upgrading PostgreSQL is a vital part of the database lifecycle. By understanding the difference between **minor** and **major** versions, using the right tool for the right job, and following best practices, you can ensure the systems remain **secure, stable, and future-ready**.

To view or add a comment, sign in.

More articles by Wayne C.

Oct 28, 2025
PostgreSQL Environment Variables Setup: Control, Customiz...
PostgreSQL is powerful on its own, but when used effectively with environment variables, it becomes a flexible...
0

Oct 27, 2025
PostgreSQL Patching: How to Stay Secure, Stable, and Ahea...
PostgreSQL is one of the most stable and secure open-source databases available. But no system is immune to bugs...
0

Oct 24, 2025
PostgreSQL Yum Installation: Deploy PostgreSQL on RHEL,...
PostgreSQL is the backbone of many mission-critical systems — but the way you install it can make or break...
0 | 1

Oct 24, 2025
PostgreSQL Source Code Installation: Build PostgreSQL fro...
PostgreSQL is renowned for its reliability, performance, and extensibility — and one of its greatest strengths lies in...

Show more

Explore content categories

Career | Productivity | Finance | Soft Skills & Emotional Intelligence | Project Management | Education

Show more

© 2025
Accessibility
Privacy Policy
Copyright Policy
Guest Controls
Language
About
User Agreement
Cookie Policy
Brand Policy
Community Guidelines